МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ

Федеральное государственное автономное
образовательное учреждение высшего образования
“Национальный исследовательский университет ИТМО”

**ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ**

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине
“Распределённые системы хранения данных”

Вариант: 85263.

Работу выполнил:
Студент группы P3311
Болорболд Аригуун
Преподаватель:
Николаев Владимир Владиславович

г. Санкт-Петербург
2025 г.

Содержимое

1 Текст задания **3**
1.1 Цель работы 3
1.2 Этап 1. Резервное копирование 3
1.3 Этап 2. Потеря основного узла 3
1.4 Этап 3. Повреждение файлов БД 3
1.5 Этап 4. Логическое повреждение данных 4

2 Выполнение **4**
2.1 Этап 1. Резервное копирование 4
2.2 Расчёт объёма резервных копий 5
2.3 Этап 2. Потеря основного узла 5
2.4 Этап 3. Повреждение файлов БД 6
2.5 Этап 4. Логическое повреждение данных 6

3 Вывод **9**

1 Текст задания

1.1 Цель работы

Настроить процедуру периодического резервного копирования базы данных, сконфигурированной в ходе выполнения лабораторной работы №2, а также разработать и отладить сценарии восстановления в случае сбоев.

Узел из предыдущей лабораторной работы используется в качестве основного. Новый узел используется в качестве резервного. Учётные данные для подключения к новому узлу выдаёт преподаватель. В сценариях восстановления необходимо использовать копию данных, полученную на первом этапе данной лабораторной работы.

1.2 Этап 1. Резервное копирование

Настроить резервное копирование с основного узла на резервный следующим образом:

- Периодические полные копии + непрерывное архивирование.
  Включить для СУБД режим архивирования `WAL`; настроить копирование `WAL` (`scp`) на резервный узел; настроить полное резервное копирование (`pg_basebackup`) по расписанию (`cron`) раз в неделю. Созданные полные копии должны сразу копироваться (`scp`) на резервный хост. Срок хранения копий на основной системе - 1 неделя, на резервной - 4 недели. По истечении срока хранения, старые архивы и неактуальные `WAL` должны автоматически уничтожаться.
- Подсчитать, каков будет объем резервных копий спустя месяц работы системы, исходя из следующих условий:
  - Средний объем новых данных в БД за сутки: 650МБ.
  - Средний объем измененных данных за сутки: 800МБ.
- Проанализировать результаты.

1.3 Этап 2. Потеря основного узла

Этот сценарий подразумевает полную недоступность основного узла. Необходимо восстановить работу СУБД на РЕЗЕРВНОМ узле, продемонстрировать успешный запуск СУБД и доступность данных.

1.4 Этап 3. Повреждение файлов БД

Этот сценарий подразумевает потерю данных (например, в результате сбоя диска или файловой системы) при сохранении доступности основного узла. Необходимо выполнить полное восстановление данных из резервной копии и перезапустить СУБД на ОСНОВНОМ узле.

Ход работы:

- Симулировать сбой:
  - удалить с диска директорию любого табличного пространства со всем содержимым.
- Проверить работу СУБД, доступность данных, перезапустить СУБД, проанализировать результаты.
- Выполнить восстановление данных из резервной копии, учитывая следующее условие:
  - исходное расположение дополнительных табличных пространств недоступно - разместить в другой директории и скорректировать конфигурацию.
- Запустить СУБД, проверить работу и доступность данных, проанализировать результаты.

1.5 Этап 4. Логическое повреждение данных

Этот сценарий подразумевает частичную потерю данных (в результате нежелательной или ошибочной операции) при сохранении доступности основного узла. Необходимо выполнить восстановление данных на ОСНОВНОМ узле следующим способом:

- Генерация файла на резервном узле с помощью `pg_dump` и последующее применение файла на основном узле.

Ход работы:

- В каждую таблицу базы добавить 2-3 новые строки, зафиксировать результат.
- Зафиксировать время и симулировать ошибку:
  - перезаписать строки любой таблицы “мусором” (INSERT, UPDATE)
- Продемонстрировать результат.
- Выполнить восстановление данных указанным способом.
- Продемонстрировать и проанализировать результат.

2 Выполнение

Узлы:

- postgres1@pg104 (Основной)
- postgres3@pg107 (Резервный)

```
TBSP_NAME_1=qmp52
TBSP_NAME_2=mzf89
```

Создадим суперпользователя:

```
CREATE ROLE admin SUPERUSER CREATEDB CREATEROLE LOGIN PASSWORD 'admin884';
```

2.1 Этап 1. Резервное копирование

Создадим `bash` скрипт, который будет настраивать непрерывное архивирование через WAL и копировать полученные архивы на резервный узел. Полное копирование происходит раз в неделю, где они хранятся на основном узле одна неделя, а в резервном — 4 недели, после которых он будет уничтожаться.

```
#!/usr/bin/env bash
DATE=$(date +%Y-%m-%d-%H-%M-%S)
BACKUP_ROOT="$(pwd)/backups"
PORT=9896
USERNAME=postgres1

pg_basebackup -D $BACKUP_ROOT -h localhost -p $PORT -U $USERNAME -Ft -z -Xs -P -v

scp -r ${BACKUP_ROOT}/* postgres1@postgres3:$(pwd)/backups/

ssh postgres1@postgres3 "find ${BACKUP_ROOT} -name '*.tar.gz' -mtime +28 -delete"
```

2.2 Расчёт объёма резервных копий

Условия:

- Средний объем новых данных в БД за сутки: 650МБ.
- Средний объем измененных данных за сутки: 800МБ.

Расчёт размера бекапов:
В сутки будет 650МБ новых данных. Размер бекапов за месяц:

$$S_{backup} = \frac{2 \cdot 0 + 650 \cdot (30 - 1)}{2} \cdot 30 = 282750\text{Мб} = 276\text{Гб} \tag{1}$$

Расчёт размеров WAL-архивов:
wal_segsize=16 Мб Объём изменений - 800 Мб -> 50 сегментов = размер wal_archive за сутки - 800Мб

$$S_{wal} = \frac{(30 - 1) \cdot 800}{2} \cdot 30 = 348000\text{Мб} = 340\text{Гб} \tag{2}$$

Итоговый размер:

$$S = S_{backup} + S_{wal} = 276 + 340 = 616\text{Гб}$$

2.3 Этап 2. Потеря основного узла

Для восстановления воссоздадим файловую структуру кластера и табличного пространства на резервном узле из нашего бекапа:

```
postgres1@postgres3:~$ mkdir wyd81
postgres1@postgres3:~$ cp -r backups/* wyd81
postgres1@postgres3:~$ chmod 750 wyd81
postgres1@postgres3:~$ cd wyd81
postgres1@postgres3:~/wyd81$ ls
16388.tar.gz  16389.tar.gz  backup_manifest  base.tar.gz  pg_wal.tar.gz
postgres1@postgres3:~/wyd81$ tar -xzf base.tar.gz
postgres1@postgres3:~/wyd81$ ls
16388.tar.gz  backup_label     base.tar.gz      log          pg_hba.conf    pg_multixact  pg_serial     pg_stat_tmp  pg_twophase    pg_xact              tablespace_map
16389.tar.gz  backup_manifest  current_logfiles  pg_commit_ts  pg_ident.conf  pg_notify     pg_snapshots  pg_subtrans  pg_wal         postgresql.auto.conf
PG_VERSION    base             global           pg_dynshmem   pg_logical     pg_replslot   pg_stat       pg_tblspc    pg_wal.tar.gz  postgresql.conf
postgres1@postgres3:~/wyd81$ tar -xzf pg_wal.tar.gz
postgres1@postgres3:~/wyd81$ tar -xzf 16388.tar.gz
postgres1@postgres3:~/wyd81$ tar -xzf 16389.tar.gz
postgres1@postgres3:~/wyd81$ echo "restore_command = cp /home/postgres1/wyd81/%f %p" >> wyd81/postgresql.conf
bash: wyd81/postgresql.conf: No such file or directory
postgres1@postgres3:~/wyd81$ echo "restore_command = cp /home/postgres1/wyd81/%f %p" >> postgresql.conf
postgres1@postgres3:~/wyd81$ touch . recovery.signal
postgres1@postgres3:~/wyd81$ cd ../
postgres1@postgres3:~$ pg_ctl -D wyd81 start
waiting for server to start....2025-09-22 09:14:01.206 GMT [385] LOG:  syntax error in file "/home/postgres1/wyd81/postgresql.conf" line 823, near token "/"
2025-09-22 09:14:01.208 GMT [385] FATAL:  configuration file "/home/postgres1/wyd81/postgresql.conf" contains errors
 stopped waiting
pg_ctl: could not start server
Examine the log output.
postgres1@postgres3:~$ nano wyd81/postgresql.conf
postgres1@postgres3:~$ pg_ctl -D wyd81 start
waiting for server to start....2025-09-22 09:15:17.578 UTC [390] LOG:  redirecting log output to logging collector process
2025-09-22 09:15:17.578 UTC [390] HINT:  Future log output will appear in directory "log".
 done
server started
```

Проверим таблицы в восстановленной БД:

```
postgres1@postgres3:~$ psql -h localhost -p 9896 -U postgres1 -d drypinknews
Password for user postgres1:
psql (16.10 (Ubuntu 16.10-0ubuntu0.24.04.1))
Type "help" for help.

drypinknews=# \d
              List of relations
 Schema |     Name      |   Type   |   Owner
--------+---------------+----------+----------
 public | table2        | table    | postgres1
 public | table2_id_seq | sequence | postgres1
(2 rows)
```

2.4 Этап 3. Повреждение файлов БД

Удаляем директорию с WAL-файлами и запустим сервер:

```
postgres1@postgres1:~/wyd81/log$ cat postgresql-2025-09-22_092818.csv
2025-09-22 09:20:18.570 UTC,,,37,,68d114d2.25,1,,2025-09-22 09:20:18 UTC,,0,LOG,00000,"ending log output to stderr",,"Future log output will go to log destination ""csvlog"".",,,,,,,"","postmaster",,0
2025-09-22 09:20:18.570 UTC,,,37,,68d114d2.25,2,,2025-09-22 09:20:18 UTC,,0,LOG,00000,"starting PostgreSQL 16.10 (Ubuntu 16.10-0ubuntu0.24.04.1) on x86_64-pc-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0,
64-bit",,,,,,,,,"","postmaster",,0
2025-09-22 09:20:18.570 UTC,,,37,,68d114d2.25,3,,2025-09-22 09:20:18 UTC,,0,LOG,00000,"listening on IPv6 address ""::1"", port 9896",,,,,,,,,"","postmaster",,0
2025-09-22 09:20:18.572 UTC,,,37,,68d114d2.25,4,,2025-09-22 09:20:18 UTC,,0,LOG,00000,"listening on IPv4 address ""127.0.0.1"", port 9896",,,,,,,,,"","postmaster",,0
2025-09-22 09:20:18.575 UTC,,,37,,68d114d2.25,5,,2025-09-22 09:20:18 UTC,,0,LOG,00000,"listening on Unix socket ""/tmp/.s.PGSQL.9896""",,,,,,,,,"","postmaster",,0
2025-09-22 09:20:18.582 UTC,,,41,,68d114d2.29,1,,2025-09-22 09:20:18 UTC,,0,LOG,00000,"database system was interrupted; last known up at 2025-09-22 08:58:02 UTC",,,,,,,,,"","startup",,0
2025-09-22 09:20:18.582 UTC,,,41,,68d114d2.29,2,,2025-09-22 09:20:18 UTC,,0,FATAL,XX000,"required WAL directory ""pg_wal"" does not exist",,,,,,,,,"","startup",,0
2025-09-22 09:20:18.583 UTC,,,37,,68d114d2.25,6,,2025-09-22 09:20:18 UTC,,0,LOG,00000,"startup process (PID 41) exited with exit code 1",,,,,,,,,"","postmaster",,0
2025-09-22 09:20:18.583 UTC,,,37,,68d114d2.25,7,,2025-09-22 09:20:18 UTC,,0,LOG,00000,"aborting startup due to startup process failure",,,,,,,,,"","postmaster",,0
2025-09-22 09:20:18.584 UTC,,,37,,68d114d2.25,8,,2025-09-22 09:20:18 UTC,,0,LOG,00000,"database system is shut down",,,,,,,,,"","postmaster",,0
```

```
postgres1@postgres1:~/wyd81$ rm -rf pg_wal
postgres1@postgres1:~/wyd81$ cd ../
postgres1@postgres1:~$ cd new/wyd81
postgres1@postgres1:~/new/wyd81$ tar -xvf ~/backups/base.tar.gz
...
postgres1@postgres1:~/new/wyd81$ cd pg_wal
postgres1@postgres1:~/new/wyd81/pg_wal$ cd archive_status
postgres1@postgres1:~/new/wyd81/pg_wal/archive_status$ ls
postgres1@postgres1:~/new/wyd81/pg_wal/archive_status$ cd ../
postgres1@postgres1:~/new/wyd81/pg_wal$ tar -xzf ~/backups/pg_wal.tar.gz
postgres1@postgres1:~/new/wyd81/pg_wal$ cd ../../
postgres1@postgres1:~/new$ mkdir qmp52
postgres1@postgres1:~/new$ mkdir mzf89
postgres1@postgres1:~/new$ cd qmp52
postgres1@postgres1:~/new/qmp52$ tar -xzf ~/backups/24576.tar.gz
postgres1@postgres1:~/new/qmp52$ cd ../mzf89
postgres1@postgres1:~/new/mzf89$ tar -xzf ~/backups/24577.tar.gz
postgres1@postgres1:~/new/mzf89$ cd ../
postgres1@postgres1:~/new$ touch wyd81/recovery.signal
postgres1@postgres1:~/new$ cat > postgresql.conf << EOF
listen_addresses = 'localhost'
port = 9896
restore_command = 'cp /home/postgres1/new/wyd81/pg_wal/%f %p
EOF
postgres1@postgres1:~/new$ pg_ctl -D wyd81 -l wyd81/server.log start
waiting for server to start.... done
server started
```

Проверим таблицы в восстановленной БД:

```
postgres1@postgres1:~/new$ psql -h localhost -U postgres1 -d postgres -p 9896
Password for user postgres1:
psql (16.10 (Ubuntu 16.10-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# \dt
        List of relations
 Schema |  Name  | Type  | Owner
--------+--------+-------+-------
 public | table1 | table | root
(1 row)

postgres=# select * from table1;
 id |   content
----+--------------
  1 | another note
  2 | another note
  3 | another note
  4 | another note
  5 | another note
(5 rows)
```

2.5 Этап 4. Логическое повреждение данных

Добавляем данные:

```
INSERT INTO table1 (content) VALUES
```

```
('note 1'),
('note 2'),
('note 3');
```

Данные в таблице в текущем состоянии:

```
postgres=# select * from table1;
 id |   content
----+-------------
  1 | another note
  2 | another note
  3 | another note
  4 | another note
  5 | another note
(5 rows)

postgres=# INSERT INTO table1 (content) VALUES
('note 1'),
('note 2'),
('note 3');
INSERT 0 3
postgres=# select * from table1;
 id |   content
----+-------------
  1 | another note
  2 | another note
  3 | another note
  4 | another note
  5 | another note
  6 | note 1
  7 | note 2
  8 | note 3
(8 rows)

postgres=#
```

Время изменений:

```
postgres=# SELECT now();
              now
-------------------------------
 2025-09-22 11:36:43.97085+00
(1 row)
```

Создание дампа:

```
#!/bin/bash

DUMPS_DIR="${HOME}/dumps"
DUMP_NAME="db-$(date +'%Y-%m-%d-%H-%M-%S').dump"

mkdir -p $DUMPS_DIR

if [ ! -f "${DUMPS_DIR}/${DUMP_NAME}" ]; then
  touch "${DUMPS_DIR}/${DUMP_NAME}"
fi

pg_dump -h localhost -p 9896 -d postgres -U postgres1 -Fc > "${DUMPS_DIR}/${DUMP_NAME}
```

Имитация порчи данных:

```
INSERT INTO table1 (content) VALUES
('trash 1'),
('trash 2'),
('trash 3');

UPDATE table1
SET content = 'bbbbbbbbbbbbbbbbb'
WHERE id > 3 AND id <= 7;
```

Итого порчи данных:

```
postgres=# \i db_spoil.sql
INSERT 0 3
UPDATE 3
postgres=# select * from table1;
 id |      content
----+-------------------
  1 | another note
  2 | another note
  3 | another note
  7 | note 2
  8 | note 3
  9 | trash1
 10 | trash2
 11 | trash3
  4 | bbbbbbbbbbbbbbbbb
  5 | bbbbbbbbbbbbbbbbb
  6 | bbbbbbbbbbbbbbbbb
(11 rows)
```

Восстановление данных:

```
#!/bin/bash

DUMPS_DIR="${HOME}/dumps"
DUMP_NAME="db-$(date +'%Y-%m-%d-%H-%M-%S').dump"

mkdir -p $DUMPS_DIR

if [ ! -f "${DUMPS_DIR}/${DUMP_NAME}" ]; then
  touch "${DUMPS_DIR}/${DUMP_NAME}"
fi

pg_dump -h localhost -p 9896 -d postgres -U postgres1 -Fc > "${DUMPS_DIR}/${DUMP_NAME}"
postgres1@postgres1:~$ cat restore_logically.sh
#!/bin/bash
```

```
DUMPS_DIR="dumps"
DUMP_NAME=$1

rsync --rsync-path=/usr/local/bin/rsync --archive ~/$DUMPS_DIR/$DUMP_NAME ~/
pg_restore -h localhost -p 9896 -d postgres -U postgres1 -c $DUMP_NAME -v
rm -rf $DUMP_NAME
```

Процесс выполнения:

```
postgres1@postgres1:~$ nano restore_logically.sh
postgres1@postgres1:~$ ./restore_logically.sh db-2025-09-22-11-43-10.dump
pg_restore: connecting to database for restore
Password:
pg_restore: dropping CONSTRAINT table1 table1_pkey
pg_restore: dropping DEFAULT table1 id
pg_restore: dropping SEQUENCE table1_id_seq
pg_restore: dropping TABLE table1
pg_restore: creating TABLE "public.table1"
pg_restore: creating SEQUENCE "public.table1_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.table1_id_seq"
pg_restore: creating DEFAULT "public.table1 id"
pg_restore: processing data for table "public.table1"
pg_restore: executing SEQUENCE SET table1_id_seq
pg_restore: creating CONSTRAINT "public.table1 table1_pkey"
pg_restore: creating ACL "SCHEMA public"
postgres1@postgres1:~$ psql -h /tmp -p 9896 -d postgres
```

Результат восстановления:

```
postgres=# select * from table1;
 id |   content
----+--------------
  1 | another note
  2 | another note
  3 | another note
  4 | another note
  5 | another note
  6 | note 1
  7 | note 2
  8 | note 3
(8 rows)
```

3 Вывод

В ходе лабораторной работы я научился восстановлению базы данных (в физическом и логическом смысле) с помощью другого резервного узла и заранее выведенных архивных файлов.